Chapter 1. A Python Q&A Session

Chương 1. Buổi trả lời câu hỏi về Python

If you're reading this book, you may already know what Python is and why it's an important tool to learn. If you don't, you probably won't be sold on Python until you've learned the language by reading the rest of this book and have done a project or two. But before we jump into details, this first chapter will briefly introduce some of the main reasons behind Python's popularity and begin sculpting a definition of the language. This takes the form of a question-and-answer session, which addresses some of the most common queries posed by beginners—like you.

Nếu bạn đang đọc cuốn sách này, có lẽ bạn đã biết Python là gì và tại sao nó lại là công cụ quan trọng cần được học. Nếu chưa biết, có lẽ bạn sẽ chưa thực sự quan tâm đến Python cho đến khi bạn tự mình học ngôn ngữ này bằng cách đọc phần còn lại của cuốn sách này, hoặc thực hiện một hoặc hai dự án liên quan đến Python. Nhưng trước khi đi vào chi tiết, chương đầu tiên này sẽ giới thiệu ngắn gọn một số lý do khiến Python trở nên phổ biến, đồng thời đưa ra định nghĩa cơ bản về ngôn ngữ này. Phần giới thiệu này được trình bày dưới dạng các câu hỏi và trả lời, nhằm giải đáp những thắc mắc phổ biến nhất của những người mới bắt đầu học Python – như bạn chẳng hạn.

Why Do People Use Python?

Tại sao mọi người lại sử dụng Python?

Because there are many programming languages to choose from, this is the usual first question of newcomers and a great place to start. Given that millions of people use Python today, there really is no way to answer this question with complete accuracy; the choice of development tools is often based on unique constraints or personal preference.

Vì có rất nhiều ngôn ngữ lập trình để lựa chọn, đây thường là câu hỏi đầu tiên mà những người mới bắt đầu học lập trình thường đặt ra. Với việc hàng triệu người trên thế giới đang sử dụng Python ngày nay, thật khó để có thể trả lời câu hỏi này một cách chính xác. Việc lựa chọn công cụ phát triển phần mềm thường phụ thuộc vào các yếu tố cụ thể hoặc sở thích cá nhân của mỗi người.

But after teaching Python to hundreds of groups and thousands of students, some common themes have emerged. The primary factors cited by Python users seem to be these:

Nhưng sau khi dạy ngôn ngữ lập trình Python cho hàng trăm nhóm học viên và hàng nghìn học sinh, tôi nhận thấy một số điểm chung trong những lý do mà người dùng Python đưa ra. Các yếu tố chính mà người dùng Python thường đề cập bao gồm:

Software quality *Chất lượng phần mềm*

For many, Python's focus on readability, coherence, and software quality in general sets it apart from other tools in the programing world. Python code is designed to be *readable*, and hence reusable and maintainable—much more so than traditional scripting languages. The uniformity of Python code makes it relatively easy to understand, even if you did not write it. In addition, Python has deep support for more advanced *software reuse* mechanisms, such as object-oriented (OO) and functional programming, that can further promote code quality.

Đối với nhiều người, việc Python chú trọng đến tính dễ đọc, tính nhất quán trong cấu trúc mã nguồn và chất lượng phần mềm nói chung đã giúp nó nổi bật so với các công cụ lập trình khác. Mã nguồn Python được thiết kế sao cho dễ đọc, do đó cũng dễ được tái sử dụng và bảo trì hơn so với các ngôn ngữ lập trình truyền thống. Tính nhất quán của mã nguồn Python giúp người khác dễ dàng hiểu được mã nguồn, ngay cả khi họ không phải là người viết ra nó. Ngoài ra, Python còn hỗ trợ tốt các mekanism tái sử dụng mã nguồn ở mức độ nâng cao, như lập trình hướng đối tượng và lập trình hàm. Những yếu tố này góp phần nâng cao chất lượng của mã nguồn Python.

Developer productivity *Năng suất làm việc của các nhà phát triển*

Python boosts developer productivity many times beyond compiled or statically typed languages. As one measure of this, Python code is typically *one-third to one-fifth* the size of equivalent C++ or Java code. That means there is less to type, less to debug, and less to maintain after the fact. Most Python programs also run immediately, without the lengthy compile and link steps required by some other tools, further boosting programmer speed.

Python giúp nâng cao năng suất làm việc của các lập trình viên gấp nhiều lần so với các ngôn ngữ lập trình được biên dịch hoặc có kiểu dữ liệu cứng nhắc. Một ví dụ minh họa cho điều này là kích thước của mã nguồn Python thường chỉ bằng một phần ba đến một phần năm so với mã nguồn C++ hoặc Java tương đương. Điều này có nghĩa là lập trình viên cần nhập mã ít hơn, việc gỡ lỗi và bảo trì cũng trở nên đơn giản hơn. Hầu hết các chương trình viết bằng Python đều có thể chạy ngay lập tức, mà không cần qua các bước biên dịch và liên kết phức tạp như các công cụ lập trình khác. Điều này giúp tăng tốc độ làm việc của các lập trình viên.

Program portability *Khả năng di chuyển chương trình sang các nền tảng khác*

Python programs generally run unchanged on *all major computer platforms*. Porting a Python program between Linux and Windows, for example, is often just a matter of copying its code between machines. Moreover, Python offers multiple options for coding portable graphical user interfaces, database access programs, web-based systems, and more. Even operating system interfaces as proprietary as program launches and directory processing are as portable in Python as they can possibly be.

Các chương trình viết bằng ngôn ngữ Python thường hoạt động tốt trên mọi nền tảng máy tính chính. Việc chuyển đổi một chương trình Python từ hệ điều hành Linux sang Windows thường chỉ đơn giản là sao chép mã nguồn của chương trình đó sang máy tính khác. Hơn nữa, Python cung cấp nhiều công cụ hữu ích để tạo ra các giao diện người dùng đồ họa, các chương trình truy cập cơ sở dữ liệu, các hệ thống dựa trên nền tảng web, v.v. Ngay cả các chức năng liên quan đến giao diện hệ điều hành, như việc khởi động chương trình hay xử lý các thư mục, cũng có thể được thực hiện một cách dễ dàng bằng Python.

Application support *Hỗ trợ việc nộp đơn xin cấp phép*

Python comes with a large collection of prebuilt and portable functionality, known as the *standard library*. This library supports an array of application-level programming tasks, from text pattern matching to network scripting. In addition, Python can be extended with both homegrown libraries and a vast collection of third-party software. As covered ahead, Python's *third-party domain* offers tools for website construction, numeric programming, AI, and much more. The NumPy extension, for instance, has elevated Python to a core tool in science, technology, engineering, and math (*STEM*), and Django and PyTorch have done similar for the web and AI.

Python được trang bị sẵn nhiều chức năng có sẵn, có thể được sử dụng ngay lập tức. Những chức năng này được gọi là thư viện tiêu chuẩn của Python. Thư viện này hỗ trợ việc thực hiện nhiều nhiệm vụ lập trình ở cấp độ ứng dụng, từ việc xử lý các mẫu văn bản cho đến việc lập trình các chương trình liên quan đến mạng máy tính. Ngoài ra, Python có thể được mở rộng bằng cách sử dụng các thư viện do chính người dùng tạo ra, cũng như nhiều phần mềm của bên thứ ba. Như đã đề cập trước đó, các công cụ của bên thứ ba giúp Python trở thành công cụ quan trọng trong lĩnh vực khoa học, công nghệ, kỹ thuật và toán học. Ví dụ, thư viện NumPy giúp Python trở thành công cụ không thể thiếu trong các lĩnh vực này. Django và PyTorch cũng đóng vai trò tương tự trong lĩnh vực phát triển web và trí tuệ nhân tạo.

Component integration *Tích hợp các thành phần cấu thành*

Python scripts can communicate with other parts of an application using a variety of integration mechanisms. Such integrations allow Python to be used as a product *customization and extension* tool. For instance, Python code can invoke compiled libraries and be run by compiled programs, interact with other components over networks, and use Android and iOS toolkits on smartphones. In fact, many Python core tools, including files, ultimately use precoded interfaces to system libraries; even if your program is all Python, it's not standalone.

Các chương trình viết bằng ngôn ngữ Python có thể giao tiếp với các thành phần khác trong ứng dụng thông qua nhiều cơ chế tích hợp khác nhau. Nhờ những cơ chế tích hợp này, Python có thể được sử dụng như công cụ để tùy biến và mở rộng chức năng của các ứng dụng. Ví dụ, mã Python có thể gọi các thư viện đã được biên dịch, được thực thi bởi các chương trình đã được biên dịch, tương tác với các thành phần khác qua mạng, và sử dụng các công cụ của hệ điều hành Android và iOS trên điện thoại thông minh. Thực tế, nhiều công cụ cốt lõi của Python, bao gồm cả các tệp tin, đều sử dụng các giao diện đã được lập trình sẵn để kết nối với các thư viện hệ thống. Ngay cả khi chương trình của bạn hoàn toàn được viết bằng Python, thì nó vẫn không thể hoạt động độc lập được.

Love of craft *Tình yêu dành cho nghề thủ công*

Because of Python's ease of use and built-in toolset, it can make the act of programming *more pleasure than chore*. Although this benefit is intangible and subjective, its effect on productivity is an important asset. People do get paid for coding Python, but many use it just for *fun*—a testimonial rare in the software field.

Nhờ tính dễ sử dụng và bộ công cụ tích hợp sẵn, Python giúp việc lập trình trở thành một công việc thú vị thay vì một nhiệm vụ nhàm chán. Mặc dù lợi ích này mang tính chủ quan, nhưng tác động của nó đối với năng suất làm việc lại rất quan trọng. Mặc dù người ta được trả lương để lập trình bằng Python, nhưng nhiều người lại sử dụng ngôn ngữ lập trình này chỉ để giải trí – điều khá hiếm gặp trong lĩnh vực phần mềm.

Of these factors, the first two—quality and productivity—are probably the most compelling benefits to most Python users; let's take a closer look at each.

Trong số các yếu tố này, hai yếu tố đầu tiên – chất lượng và năng suất – có lẽ là những lợi ích quan trọng nhất đối với hầu hết người dùng Python. Hãy cùng xem xét từng yếu tố một một cách kỹ lưỡng hơn.

Software Quality Chất lượng phần mềm

By design, Python has a deliberately simple and readable syntax and a highly consistent programming model. As a slogan at an early Python conference attested, the net result is that Python seems to *fit your brain*—that is, features of the language interact in consistent and limited ways and follow naturally from a small set of core concepts. This makes the language easier to learn, understand, and remember. In practice, Python programmers do not need to constantly refer to manuals when reading or writing code; it generally just makes sense.

Theo thiết kế ban đầu, Python có cú pháp đơn giản và dễ hiểu, cùng với mô hình lập trình rất nhất quán. Như lời được nói trong một hội nghị về Python vào thời điểm đó, Python thực sự phù hợp với cách suy nghĩ của con người: các tính năng của ngôn ngữ này tương tác với nhau một cách nhất quán và có quy luật, đồng thời xuất phát từ một tập hợp các khái niệm cơ bản. Điều này giúp ngôn ngữ này dễ học, dễ hiểu và dễ nhớ hơn. Trong thực tế, các lập trình viên Python không cần phải liên tục tham khảo sách hướng dẫn khi viết mã; mọi thứ đều khá dễ hiểu một cách tự nhiên.

By philosophy, Python adopts a somewhat minimalist mindset. This means that although there are usually multiple ways to accomplish a coding task, there is usually just one obvious way, a few less obvious alternatives, and a small set of coherent interactions throughout. Moreover, Python usually doesn't make arbitrary decisions for you; when interactions are ambiguous, explicit intervention is preferred over "magic." In the Python way of thinking, explicit is better than implicit, and simple is better than complex.

Theo triết lý lập trình của mình, Python áp dụng cách tiếp cận tối giản hóa mọi thứ. Điều này có nghĩa là, mặc dù thường có nhiều cách khác nhau để hoàn thành một nhiệm vụ lập trình, nhưng thường chỉ có một cách là rõ ràng nhất, vài cách khác ít rõ ràng hơn, cùng với một số quy tắc nhất quán trong cách thức tương tác giữa các thành phần của chương trình. Hơn nữa, Python không tự ý đưa ra các quyết định một cách tùy tiện; khi có sự mơ hồ trong cách thức tương tác giữa các thành phần của chương trình, việc can thiệp một cách rõ ràng sẽ được ưu tiên thay vì việc sử dụng các "mánh khóe" kỳ lạ. Theo cách suy nghĩ của Python, việc làm mọi thứ một cách rõ ràng, đơn giản sẽ tốt hơn việc làm mọi thứ một cách mơ hồ, phức tạp.

Beyond such design themes, Python includes tools such as modules and object-oriented programming (OOP) that naturally promote code reusability in skilled hands of the sort you're about to acquire. And because Python is focused on quality, most Python programmers naturally are too; this can be a crucial advantage when it's time to use someone else's Python code.

Ngoài những yếu tố thiết kế nói trên, Python còn cung cấp nhiều công cụ hữu ích như các mô-đun và phương pháp lập trình hướng đối tượng (OOP). Những công cụ này giúp tăng khả năng tái sử dụng mã nguồn một cách hiệu quả, đặc biệt trong tay những lập trình viên có kỹ năng cao. Vì Python luôn coi trọng chất lượng, nên hầu hết các lập trình viên Python cũng đều có tư duy tương tự. Đây chính là lợi thế lớn khi cần sử dụng mã nguồn Python do người khác viết ra.

Developer Productivity Năng suất của các nhà phát triển phần mềm

If you've worked in the software field, you know that it can be a dynamic and bumpy ride. During the great internet boom of the mid-to-late 1990s, it was difficult to find enough programmers to implement software projects; developers were asked to implement systems as fast as the internet evolved. In later eras of economic recession and layoffs, the picture shifted; programming staffs were often asked to accomplish the same tasks with even fewer people.

Nếu bạn từng làm việc trong lĩnh vực phần mềm, bạn chắc hẳn biết rằng đây là một lĩnh vực đầy biến động và khó lường. Trong thời kỳ bùng nổ của Internet vào giữa và cuối những năm 1990, rất khó tìm đủ lập trình viên để thực hiện các dự án phần mềm. Các nhà phát triển phải hoàn thành công việc càng nhanh càng tốt, để theo kịp sự phát triển của Internet. Trong các giai đoạn suy thoái kinh tế sau đó, số lượng lập trình viên càng ngày càng ít đi, trong khi yêu cầu về việc hoàn thành các nhiệm vụ vẫn không thay đổi.

In both of these scenarios, Python has shined as a tool that allows programmers to get more done with less effort. It is explicitly optimized for *speed of development*—its simple syntax, dynamic typing, lack of compile steps, and built-in toolset allow programmers to develop programs in a fraction of the time needed when using some other tools.

Trong cả hai trường hợp này, Python đều thể hiện vai trò của mình như một công cụ giúp các lập trình viên hoàn thành công việc một cách hiệu quả hơn, với nỗ lực tối thiểu. Python được tối ưu hóa để nâng cao tốc độ phát triển phần mềm: cú pháp đơn giản, khả năng xác định kiểu dữ liệu dinhamic, không cần qua các bước biên dịch, cùng bộ công cụ tích hợp sẵn giúp các lập trình viên hoàn thành công việc nhanh hơn nhiều so với việc sử dụng các công cụ khác.

The net effect is that Python typically increases developer productivity many times beyond the levels supported by traditional languages and often makes the impossible possible. That's good news in both boom and bust times, and everywhere the software industry goes in between.

Tổng thể mà nói, Python giúp nâng cao năng suất làm việc của các lập trình viên gấp nhiều lần so với các ngôn ngữ lập trình truyền thống. Nó thường giúp biến những điều tưởng chừng bất khả thi thành hiện thực. Đó là tin tốt trong mọi hoàn cảnh, dù là thời kỳ thị trường phát triển mạnh hay suy thoái.

Is Python a "Scripting Language"?

Liệu Python có phải là một "ngôn ngữ lập trình kịch bản" không?

Maybe. Python is often applied in scripting roles, but not always. It's regularly called an *object-oriented scripting language*—a definition that blends support for OOP with an orientation toward scripting contexts, but this may be too narrow and dismissive. If pressed for a one-liner, Python is probably better known as:

Có lẽ vậy. Python thường được sử dụng trong các ứng dụng lập trình kịch bản, nhưng không phải lúc nào cũng vậy. Python thường được coi là ngôn ngữ lập trình hướng đối tượng; định nghĩa này kết hợp khả năng hỗ trợ lập trình hướng đối tượng với tính năng phù hợp để sử dụng trong các ứng dụng kịch bản. Tuy nhiên, định nghĩa này có vẻ hơi hạn chế và thiếu chính xác. Nếu phải mô tả Python bằng một câu ngắn gọn, có lẽ Python nên được biết đến như sau:

> *A general-purpose programming language that blends procedural, functional, and object-oriented paradigms and accelerates software development by reducing complexity.*
>
> *Đó là một ngôn ngữ lập trình đa năng, kết hợp các phương pháp lập trình theo hướng thủ tục, hướng chức năng và*

hướng đối tượng. Ngôn ngữ này giúp tăng tốc quá trình phát triển phần mềm bằng cách giảm độ phức tạp của các chương trình.

That may not fit on a t-shirt quite as well, but it captures both the richness and scope of today's Python.

Có lẽ điều đó sẽ không phù hợp lắm khi được in lên áo phông. Nhưng nó vẫn giúp thể hiện được sự phong phú và tính đa dạng của ngôn ngữ lập trình Python ngày nay.

Nevertheless, the term *scripting* seems to have stuck to Python like glue, perhaps as a contrast with the larger efforts required by some other tools. For example, some people use the word "script" instead of "program" to describe a Python code file, because it seems simpler and less formal to code. More usefully, others reserve "script" for a top-level file, and "program" for a more sophisticated multifile application, both of which are common in Python.

Tuy nhiên, thuật ngữ "script" dường như đã gắn liền với Python như một thứ keo, có lẽ để phân biệt với những công cụ khác đòi hỏi nhiều công sức hơn. Ví dụ, một số người sử dụng từ "script" thay vì "program" để mô tả một tệp mã Python, vì nó có vẻ đơn giản và ít trang trọng hơn. Ngoài ra, một số người sử dụng "script" để chỉ một tệp cấp cao, và "program" cho một ứng dụng phức tạp nhiều tệp, điều này phổ biến trong Python.

Because the term *scripting language* has so many different meanings to different observers, though, some would prefer that it not be applied to Python at all. In fact, people tend to make three very different assumptions when they hear Python labeled as such, some of which are more useful than others:

Tuy nhiên, vì thuật ngữ "ngôn ngữ lập trình kịch bản" có nhiều ý nghĩa khác nhau tùy theo góc nhìn của từng người, nên một số người thích không sử dụng thuật ngữ này để chỉ Python. Thực tế, mọi người thường đưa ra ba giả định khác nhau khi nghe nói về Python. Trong số đó, một số giả định có ích hơn những giả định khác.

Shell tools *Các công cụ của Shell*

Sometimes when people hear Python described as a scripting language, they think it means that Python is a tool for coding operating-system-oriented scripts. Such programs are often launched from console command lines and perform tasks such as processing text files and launching other programs.

Đôi khi, khi nghe nói rằng Python là một ngôn ngữ lập trình kịch bản, mọi người thường nghĩ rằng Python chỉ là công cụ dùng để viết các chương trình liên quan đến hệ điều hành. Những chương trình như vậy thường được khởi chạy từ dòng lệnh trên console, và chúng thực hiện các nhiệm vụ như xử lý các tệp văn bản hay khởi chạy các chương trình khác.

As you'll learn ahead, Python programs can and do serve such roles, but this is just one of dozens of common Python application domains. It is not just a better shell-script language.

Như bạn sẽ tìm hiểu sau này, các chương trình viết bằng Python hoàn toàn có thể đảm nhận những vai trò như vậy. Tuy nhiên, đây chỉ là một trong hàng chục lĩnh vực ứng dụng phổ biến của Python mà thôi. Python không đơn thuần chỉ là một ngôn ngữ lập trình dùng để viết các script.

Control language *Ngôn ngữ điều khiển*

To others, scripting refers to a "glue" layer used to control and direct (i.e., script) other application components. As noted earlier, Python programs are indeed often deployed in the context of larger applications. For instance, to test hardware devices, Python programs may call out to components that give low-level access to a device or port. Similarly, programs may run bits of Python code at strategic points to support end-user product customization without the need to ship and recompile the entire system's source code.

Đối với những người khác, khái niệm "scripting" là chỉ lớp "kết nối" được sử dụng để kiểm soát và điều khiển các thành phần khác của ứng dụng. Như đã đề cập trước đó, các chương trình viết bằng ngôn ngữ Python thường được sử dụng trong các ứng dụng lớn hơn. Ví dụ, để kiểm thử các thiết bị phần cứng, các chương trình Python có thể gọi các thành phần cung cấp quyền truy cập ở cấp độ thấp vào các thiết bị hoặc cổng kết nối tương ứng. Tương tự, các chương trình có thể chạy các đoạn mã Python ở những điểm cụ thể nhằm hỗ trợ việc cá nhân hóa sản phẩm cho người dùng cuối, mà không cần phải phát hành và biên dịch lại toàn bộ mã nguồn của hệ thống.

Python's simplicity makes it a naturally flexible control tool. Technically, though, this is also just a common Python role; many (and perhaps most) Python programmers code standalone scripts without ever using or knowing about any integrated components. It is not just a control language.

Sự đơn giản của Python khiến nó trở thành công cụ điều khiển linh hoạt. Tuy nhiên, về mặt kỹ thuật, đây chỉ là một trong những chức năng thông thường của Python mà thôi. Nhiều lập trình viên Python viết các chương trình độc lập, mà không cần sử dụng hay biết gì về các thành phần tích hợp khác. Python không chỉ đơn thuần là một ngôn ngữ lập trình dùng để điều khiển mà thôi.

Ease of use *Dễ sử dụng*

Probably the best use of the term *scripting language* is to refer to a relatively simple language used for coding tasks quickly. This is especially true when the term is applied to Python, which allows much faster program development than compiled languages like C++ or larger languages like Java. Its rapid development cycle fosters an exploratory, incremental mode of programming that has to be experienced to be appreciated.

Có lẽ cách sử dụng phù hợp nhất cho thuật ngữ "ngôn ngữ lập trình" chính là để chỉ những ngôn ngữ tương đối đơn giản, được dùng để thực hiện các công việc lập trình một cách nhanh chóng. Điều này đặc biệt đúng trong trường hợp của Python: ngôn ngữ này giúp việc phát triển chương trình diễn ra nhanh hơn nhiều so với các ngôn ngữ biên dịch như C++ hay các ngôn ngữ lớn hơn như Java. Chu kỳ phát triển nhanh chóng của Python góp phần tạo ra phong cách lập trình dựa trên việc thử nghiệm và cải tiến từng bước một; đây là điều mà chỉ khi trải nghiệm thực tế mới có thể hiểu được trọn vẹn.

Don't be fooled, though—Python is not just for simple tasks. Rather, it makes tasks simple by its ease of use and flexibility. Python has an approachable feature set, but it allows programs to scale up in sophistication as needed. Because of that, it is commonly used for both quick tactical tasks and longer-term strategic development.

Đừng nhầm lẫn nhé: Python không chỉ phù hợp để thực hiện những công việc đơn giản. Thực chất, Python giúp các công việc trở nên đơn giản hơn nhờ vào tính dễ sử dụng và tính linh hoạt của mình. Python có bộ tính năng khá đơn giản, nhưng lại cho phép các chương trình được mở rộng và trở nên phức tạp hơn tùy theo nhu cầu sử dụng. Chính vì vậy, Python thường được sử dụng cả cho những công việc cần thực hiện nhanh chóng, lẫn những dự án phát triển dài hạn.

So, is Python a scripting language or not? It depends on whom you ask (and perhaps when you ask them). In general, the term *scripting* is best reserved for the rapid and flexible mode of development that Python supports, rather than a particular application domain or limiting label.

Vậy, Python có phải là ngôn ngữ lập trình dạng kịch bản hay không? Điều này còn tùy thuộc vào người được hỏi (và có lẽ cả thời điểm hỏi nữa). Nhìn chung, thuật ngữ "ngôn ngữ lập trình dạng kịch bản" nên được dùng để mô tả cách phát triển phần mềm nhanh chóng và linh hoạt mà Python hỗ trợ, chứ không phải để chỉ một lĩnh vực ứng dụng cụ thể nào đó.

On a related note, people also sometimes call Python an *interpreted language* to distinguish it from languages like C and C++. While this is also sometimes meant to pigeonhole, it's also easier to dismiss: there are many implementations of Python today, spanning the spectrum from traditional interpreters to traditional compilers, so "interpreted" doesn't apply. The clearer distinction may be that Python is *dynamically typed*, not statically typed like languages that are normally compiled. As you'll soon learn, this accounts for much of the power that Python brings to development tasks.

Liên quan đến điều này, đôi khi người ta cũng gọi Python là ngôn ngữ được giải thích trực tiếp, nhằm phân biệt nó với các ngôn ngữ như C và C++. Mặc dù cách phân loại này có phần hạn chế, nhưng cũng dễ hiểu thôi: Ngày nay, có rất nhiều cách để triển khai Python, từ các chương trình được giải thích trực tiếp cho đến các chương trình được biên dịch theo kiểu truyền thống. Vì vậy, việc gọi Python là ngôn ngữ được giải thích trực tiếp không còn phù hợp nữa. Sự khác biệt rõ ràng hơn có lẽ là Python thuộc loại ngôn ngữ có kiểu dữ liệu động, khác với các ngôn ngữ thường được biên dịch trước khi chạy, vốn thuộc loại ngôn ngữ có kiểu dữ liệu tĩnh. Như bạn sẽ sớm hiểu, đây chính là yếu tố giúp Python trở thành công cụ mạnh mẽ trong việc phát triển phần mềm.

OK, but What’s the Downside?

Được rồi, nhưng điểm yếu của nó là gì?

The only universally recognized downside to Python that has emerged over its more than three-decade tenure may also be an inevitable trade-off for its ease of use: Python’s *execution speed* may not always be as fast as that of fully compiled and lower-level languages such as C and C++. Though relatively rare today, you may still occasionally need to get “closer to the iron” for some tasks by using languages that are more directly mapped to the underlying hardware.

Điểm yếu duy nhất của Python mà được công nhận rộng rãi trong suốt hơn ba thập kỷ tồn tại của nó, có lẽ cũng là một cái giá phải trả cho tính dễ sử dụng của nó. Tốc độ thực thi của Python có thể không nhanh bằng các ngôn ngữ được biên dịch hoàn toàn và có cấp độ thấp hơn như C và C++. Mặc dù hiện nay điều này khá hiếm gặp, nhưng đôi khi vẫn cần phải sử dụng các ngôn ngữ cho phép tiếp cận trực tiếp với phần cứng, để hoàn thành một số nhiệm vụ cụ thể.

We’ll talk about implementation concepts in Chapter 2, but in short, the most-used versions of Python today compile (i.e., translate) source code statements to an intermediate format known as *bytecode* and then interpret the bytecode. Bytecode provides portability, as it is a platform-independent format. However, because Python is not commonly compiled all the way down to binary *machine code* (e.g., instructions for an Intel or ARM chip in your PC or phone), some programs will run more slowly in Python than in a fully compiled language like C.

Chúng ta sẽ thảo luận về các khái niệm liên quan đến việc triển khai chương trình trong Chương 2. Nhìn chung, các phiên bản Python phổ biến ngày nay đều chuyển đổi các đoạn mã nguồn thành một định dạng trung gian gọi là bytecode, sau đó mới thực hiện các lệnh trong bytecode đó. Định dạng bytecode giúp cho việc chạy chương trình trên nhiều nền tảng khác nhau trở nên dễ dàng hơn, vì đây là định dạng không phụ thuộc vào nền tảng cụ thể. Tuy nhiên, vì Python thường không được chuyển đổi hoàn toàn thành mã máy nhị phân (tức là các lệnh dành riêng cho chip Intel hoặc ARM trong máy tính hoặc điện thoại của bạn), nên một số chương trình sẽ chạy chậm hơn khi được viết bằng Python so với khi được viết bằng ngôn ngữ lập trình được biên dịch hoàn toàn như C.

Whether you will ever *care* about this execution speed difference depends on what kinds of programs you write. Python is regularly optimized, and Python code runs fast enough by itself in most application domains. Furthermore, whenever you do something “real” in a Python script, like processing a file or constructing a graphical user interface (*GUI*), your program will actually run at C speed, because such tasks are immediately dispatched to compiled C code inside the Python interpreter.

Việc bạn có quan tâm đến sự khác biệt về tốc độ thực thi này hay không phụ thuộc vào loại chương trình mà bạn viết. Python thường xuyên được tối ưu hóa, vì vậy mã Python hoạt động khá nhanh trong hầu hết các ứng dụng. Hơn nữa, mỗi khi bạn thực hiện những công việc “thực sự quan trọng” trong chương trình Python, chẳng hạn như xử lý tệp tin hoặc tạo giao diện người dùng đồ họa, chương trình của bạn sẽ hoạt động với tốc độ tương đương với mã C. Bởi vì những công việc này sẽ được chuyển thành mã C để thực hiện ngay lập tức bởi trình thông dịch Python.

And really, Python’s speed is a bit of a red herring today: in truth, Python *is* commonly used in domains that require optimal execution speeds. Numeric programming and computer games, for example, often need at least their core number-crunching components to run at C speed (or better), but are frequently coded in Python nevertheless.

Thực ra, tốc độ xử lý của Python hiện nay không còn là yếu tố quan trọng nữa. Trên thực tế, Python thường được sử dụng trong những lĩnh vực đòi hỏi tốc độ xử lý cao. Ví dụ, trong lập trình số và phát triển trò chơi điện tử, các thành phần liên quan đến việc xử lý dữ liệu số thường cần được thực hiện với tốc độ tương đương hoặc cao hơn tốc độ của ngôn ngữ C. Tuy nhiên, người ta vẫn thường sử dụng Python để lập trình cho các ứng dụng này.

There are multiple ways to achieve speed in Python when it counts. For instance, systems such as *PyPy* compile bytecode further as your program runs; *Cython* allows you to code in a C-and-Python hybrid that's compiled in full; and crucial code can be split off to *compiled extensions* linked into Python for use in scripts. As an example, the *NumPy* numeric-programming extension combines compiled and optimized libraries with the Python language, and in the process turns Python into a numeric programming tool that is simultaneously efficient and easy to use. Indeed, NumPy Python code regularly achieves speed parity with Fortran and C++, without their added complexities.

Trong Python, có nhiều cách khác nhau để tăng tốc độ thực thi chương trình. Ví dụ, các công cụ như PyPy sẽ biên dịch mã nguồn thành mã bytecode trong quá trình chương trình đang chạy; Cython cho phép lập trình bằng ngôn ngữ kết hợp giữa C và Python, sau đó mã được biên dịch hoàn chỉnh. Những đoạn mã quan trọng có thể được tách ra thành các tệp riêng biệt, sau đó được biên dịch và kết hợp vào Python để sử dụng trong các chương trình. Ví dụ về điều này là thư viện NumPy: thư viện này kết hợp các thư viện đã được biên dịch và tối ưu hóa với ngôn ngữ Python. Nhờ đó, Python trở thành công cụ lập trình số hiệu quả và dễ sử dụng. Thực tế, mã nguồn viết bằng NumPy thường có tốc độ tương đương với mã nguồn viết bằng Fortran và C++, mà không cần phải đối mặt với những độ phức tạp thêm vào.

More fundamentally, though, execution speed is not the only priority in most software development. Python's *speed-of-development* gain is often far more important than any speed-of-execution loss, especially given modern computer speeds—and modern computer deadlines. Naturally, Python has other *potential* downsides, including its frequent changes and convolutions, but these are subjective calls best made after you've had a chance to vet them in this book.

Tuy nhiên, về bản chất, tốc độ thực thi không phải là yếu tố duy nhất cần được quan tâm trong quá trình phát triển phần mềm. Lợi thế về tốc độ phát triển của Python thường quan trọng hơn nhiều so với những thiệt hại về tốc độ thực thi, đặc biệt khi xét đến tốc độ xử lý của các máy tính hiện đại và các thời hạn yêu cầu trong quá trình phát triển phần mềm. Tất nhiên, Python cũng có một số nhược điểm, chẳng hạn như việc mã nguồn thường xuyên thay đổi. Nhưng những nhược điểm này chỉ nên được đánh giá sau khi bạn đã có cơ hội tìm hiểu kỹ về Python thông qua cuốn sách này.

Who Uses Python Today?

Ai sử dụng Python ngày nay?

Because Python is a free and open source software (*FOSS*) tool, an accurate count of its user base is impossible—there are no license registrations to tally. Moreover, Python has been automatically included with countless Linux distributions, Macintosh computers, and a wide range of products and hardware, further clouding the user-base picture.

Vì Python là một công cụ phần mềm mã nguồn mở, nên việc ước lượng chính xác số lượng người dùng là điều không thể. Không có bất kỳ hệ thống đăng ký giấy phép nào để ghi nhận số lượng người dùng. Hơn nữa, Python được tích hợp sẵn trong vô số bản phân phối Linux, máy tính Macintosh, cũng như nhiều loại sản phẩm và thiết bị khác. Điều này khiến việc xác định chính xác số lượng người dùng trở nên khó khăn hơn.

In general, though, Python enjoys a very large user base and an active developer community. It is generally considered to be among the *top 5* most widely used programming languages in the world today (and for true sports fans, often weighs in at #1). Because Python has been broadly used for *over three decades*, it's also very stable and robust.

Nhìn chung, Python sở hữu lượng người dùng rất lớn và cộng đồng các lập trình viên tích cực. Ngôn ngữ lập trình này thường được xếp vào top 5 ngôn ngữ lập trình được sử dụng phổ biến nhất trên thế giới hiện nay (và đối với những người yêu thích thể thao, Python thường đứng ở vị trí số 1). Vì đã được sử dụng rộng rãi trong hơn ba thập kỷ, Python cũng rất ổn định và bền vững.

Because of all this, Python is regularly applied in real revenue-generating products by real *companies*. While user lists are prone to change, a list of notable and known companies using Python today reads like a who's who of the software field: Google, Intel, Disney, YouTube, Industrial Light & Magic, Red Hat, NASA, Eve Online, Seagate, JPL, Hewlett-Packard, JP Morgan Chase, Dropbox, ESRI, Instagram, Spotify, Pinterest, Reddit, Microsoft, Netflix, and many more.

Vì những lý do trên, Python thường được các công ty thực sự sử dụng trong việc tạo ra các sản phẩm mang lại doanh thu thực tế. Mặc dù danh sách các công ty sử dụng Python có thể thay đổi theo thời gian, nhưng những công ty nổi tiếng và có uy tín hiện nay đều sử dụng Python: Google, Intel, Disney, YouTube, Industrial Light & Magic, Red Hat, NASA, Eve Online, Seagate, JPL, Hewlett-Packard, JP Morgan Chase, Dropbox, ESRI, Instagram, Spotify, Pinterest, Reddit, Microsoft, Netflix, và nhiều công ty khác nữa.

More broadly, Python is deployed by most organizations developing software today in either strategic or tactical roles and regularly serves as the tool of choice in computer science education. It's not just for one thing, it's for everything.

Nói một cách tổng quát hơn, Python được hầu hết các tổ chức phát triển phần mềm ngày nay sử dụng, dù trong các mục đích chiến lược hay chiến thuật. Đồng thời, Python cũng thường được coi là công cụ lý tưởng để giảng dạy khoa học máy tính. Python không chỉ được sử dụng cho một mục đích cụ thể nào đó, mà có thể được áp dụng vào mọi lĩnh vực.

For a sampling of additional Python users and applications that's hopefully more up to date than a book can ever be, try the following pages at Python's site: Python Success Stories, Applications for Python, and Quotes about Python. As usual, you may also be able to uncover other Python roles of interest with a web search in your local browser or app.

Để tìm hiểu thêm về các người dùng và ứng dụng liên quan đến Python, những thông tin này chắc chắn sẽ cập nhật hơn nhiều so với những gì có thể được ghi trong sách. Bạn có thể truy cập các trang sau trên trang web chính thức của Python: Python Success Stories, Applications for Python, và Quotes about Python. Như mọi khi, bạn cũng có thể tìm thêm nhiều thông tin hữu ích về Python bằng cách tìm kiếm trên trình duyệt hoặc ứng dụng trên thiết bị của mình.

What Can I Do with Python?

Tôi có thể làm gì với Python?

Commercial applications may be compelling, but people also use Python for all sorts of real-world, day-in/day-out tasks, whether for profit, need, hobby, or fun. In fact, as a general-purpose language, Python's roles are virtually unlimited: you can use it for everything from gaming and website development to robotics and content management.

Các ứng dụng thương mại có thể rất hấp dẫn, nhưng mọi người cũng sử dụng Python cho đủ loại công việc trong đời sống hàng ngày, dù là vì mục đích kinh doanh, nhu cầu cá nhân, sở thích hay chỉ để giải trí. Thực tế, với tư cách là một ngôn ngữ lập trình đa năng, Python có thể được sử dụng trong mọi lĩnh vực: từ việc phát triển trò chơi và trang web, cho đến lĩnh vực robot và quản lý nội dung.

That said, Python roles seem to fall into a few broad categories. The next few sections survey some of Python's most common application domains today, as well as prominent tools used in each domain.

Tuy nhiên, các vai trò của Python có vẻ thuộc vào một vài nhóm chính. Các phần tiếp theo sẽ trình bày một số lĩnh vực ứng dụng phổ biến nhất của Python ngày nay, cùng với các công cụ quan trọng được sử dụng trong từng lĩnh vực đó.

Two notes up front: first, we won't be able to explore these tools in any depth either here or in this book at large. NumPy and Django, for example, are book-length topics on their own, and our goal in this book is to learn the *Python* code you will be writing to use systems like these. Second, apologies in advance to the many other tools omitted here only for space; if you are interested in any of the following topics, please search online for more tools and resources.

Xin nói rõ hai điểm trước khi bắt đầu: Thứ nhất, chúng ta sẽ không thể tìm hiểu sâu về các công cụ này, dù là ở đây hay trong toàn bộ nội dung cuốn sách này. Ví dụ, NumPy và Django là những công cụ phức tạp đến mức cần có cả những cuốn sách riêng để giải thích chi tiết về chúng. Mục tiêu của cuốn sách này là giúp bạn hiểu cách viết mã Python để sử dụng các công cụ như vậy. Thứ hai, xin chân thành xin lỗi những ai muốn tìm hiểu về các công cụ khác nhưng lại không thể được đề cập trong cuốn sách này do hạn chế về không gian. Nếu bạn quan tâm đến bất kỳ công cụ nào trong số này, xin vui lòng tìm kiếm thêm thông tin trên mạng.

Systems Programming Lập trình hệ thống

Python's built-in interfaces to operating-system services make it ideal for writing portable and maintainable system-administration utilities—sometimes called *shell* or *command-line* tools, though they may be used in numerous ways. Python programs can search files and directory trees, launch and configure other programs, do parallel processing with processes, threads, and coroutines, and more.

Các giao diện tích hợp sẵn của Python với các dịch vụ của hệ điều hành khiến nó trở thành công cụ lý tưởng để tạo ra các chương trình quản trị hệ thống có khả năng chạy trên nhiều nền tảng và dễ bảo trì. Những chương trình này thường được gọi là các công cụ dòng lệnh hoặc shell tools; tuy nhiên, chúng có thể được sử dụng trong nhiều mục đích khác nhau. Các chương trình viết bằng Python có thể tìm kiếm trong các tệp tin và thư mục, khởi chạy và cấu hình các chương trình khác, thực hiện các thao tác xử lý song song bằng cách sử dụng các tiến trình, luồng và coroutine, v.v.

Python's standard library comes with Portable Operating System Interface (*POSIX)* bindings and support for all the usual OS tools, including environment variables, files, sockets, pipes, processes, threads, regular-expression pattern matching, command-line arguments, standard-stream interfaces, shell-command launchers, filename expansion, ZIP file utilities, and XML, JSON, and CSV parsers. In addition, the bulk of Python's system interfaces are designed to be portable; for example, a script that copies directory trees typically runs unchanged on all platforms that host Python.

Thư viện tiêu chuẩn của Python bao gồm các công cụ hỗ trợ giao diện với hệ điều hành kiểu POSIX, đồng thời hỗ trợ tất cả các công cụ thông thường của hệ điều hành như: các biến môi trường, tệp tin, kết nối mạng, các tiến trình, luồng xử lý, chức năng so khớp mẫu chuỗi, các tham số dòng lệnh, giao diện luồng dữ liệu tiêu chuẩn, các lệnh để khởi chạy chương trình từ dòng lệnh, chức năng mở rộng tên tệp tin, các công cụ xử lý tệp ZIP, cũng như các công cụ phân tích dữ liệu dạng XML, JSON và CSV. Ngoài ra, hầu hết các giao diện hệ thống của Python đều được thiết kế sao cho có thể hoạt động trên nhiều nền tảng khác nhau. Ví dụ, một chương trình dùng để sao chép cấu trúc thư mục sẽ hoạt động bình thường trên mọi nền tảng hỗ trợ Python.

GUIs and UIs Giao diện người dùng và giao diện đồ họa

Python's simplicity and rapid turnaround also make it a good match for GUI programming on devices of all kinds. For instance, Python comes with a standard object-oriented interface to the Tk GUI toolkit called *Tkinter* (and `tkinter` in code), which allows Python programs to implement portable GUIs with a native look and feel. Python/Tkinter GUIs run unchanged on Windows, macOS, and Linux, and on Android with the help of a freeware app today (see Appendix A).

Sự đơn giản và tốc độ xử lý nhanh của Python khiến nó trở thành công cụ lý tưởng để phát triển các giao diện người dùng trên mọi loại thiết bị. Chẳng hạn, Python có giao diện lập trình hướng đối tượng tiêu chuẩn để tương tác với công cụ tạo giao diện Tkinter. Nhờ đó, các chương trình viết bằng Python có thể tạo ra các giao diện người dùng có vẻ ngoài và cảm giác sử dụng tương tự như các giao diện gốc của các hệ điều hành tương ứng. Các giao diện người dùng được tạo bằng Python/Tkinter hoạt động tốt trên Windows, macOS, Linux, và cả Android, nhờ vào sự hỗ trợ của các ứng dụng miễn phí hiện nay (xem Phụ lục A).

In addition, third-party tools offer other routes to portable UIs—including both traditional GUIs like *Kivy*, *BeeWare*'s *Toga*, *PyQt*, and *wxPython*; and web-browser based solutions like *Django*, *Flask*, and *WebAssembly*. If your app, like most, must interact with users, Python has multiple ways to write once and run everywhere.

Ngoài ra, các công cụ của bên thứ ba còn cung cấp nhiều cách khác nhau để tạo giao diện người dùng di động. Đó có thể là các giao diện GUI truyền thống như Kivy, Toga của BeeWare, PyQt và wxPython; hoặc các giải pháp dựa trên trình duyệt web như Django, Flask và WebAssembly. Nếu ứng dụng của bạn, giống như hầu hết các ứng dụng khác, cần tương tác với người dùng, thì Python cung cấp nhiều cách để viết mã một lần và chạy nó trên mọi nền tảng.

Internet and Web Scripting

Internet và Lập trình Web

Python comes with standard internet modules that allow programs to perform a wide variety of networking tasks, in both client and server modes. Scripts can communicate over sockets; extract form information sent to server-side CGI web scripts; transfer files by FTP; parse and generate XML and JSON documents; compose and send, and receive and parse email; fetch web pages by URLs and parse their HTML; and more.

Python có sẵn các mô-đun kết nối mạng tiêu chuẩn, giúp các chương trình có thể thực hiện nhiều loại tác vụ mạng khác nhau, cả ở chế độ máy khách lẫn máy chủ. Các chương trình được viết bằng Python có thể giao tiếp qua các kết nối socket; trích xuất thông tin từ các biểu mẫu được gửi đến các chương trình web CGI ở phía máy chủ; truyền tệp tin qua FTP; phân tích và tạo ra các tài liệu dạng XML và JSON; soạn thảo, gửi, nhận và phân tích email; truy cập các trang web thông qua URL và phân tích nội dung HTML của chúng. Và còn nhiều tính năng khác nữa.

In addition, a large collection of third-party tools are available on the web for doing internet programming in Python. For example, web-development frameworks, such as *Django*, *Flask*, *TurboGears*, and *Zope*, support construction of full-featured and production-quality websites with Python. Many of these include tools like object-relational mappers, server-side scripting and templating, and AJAX support, to provide complete and enterprise-level web development solutions. The *Pyjamas* Python-to-JavaScript transpiler; the *Beautiful Soup* HTML content extractor; and the *WebAssembly*, *Pyodide*, *py2wasm*, and *PyScript* Python-in-the-browser enablers provide even more web possibilities.

Ngoài ra, còn có rất nhiều công cụ từ các bên thứ ba có sẵn trên mạng để phát triển các ứng dụng web bằng ngôn ngữ Python. Ví dụ, các framework phát triển web như Django, Flask, TurboGears và Zope giúp tạo ra các trang web chất lượng cao bằng Python. Nhiều công cụ trong số này bao gồm các công cụ hỗ trợ việc xử lý dữ liệu, viết mã phía máy chủ, tạo mẫu trang web, và hỗ trợ công nghệ AJAX. Những công cụ như Pyjamas (chuyển đổi mã Python sang JavaScript), Beautiful Soup (công cụ trích xuất nội dung HTML), cùng các công cụ hỗ trợ công nghệ WebAssembly, Pyodide, py2wasm và PyScript giúp mở rộng thêm nhiều khả năng trong việc phát triển web.

Component Integration Tích hợp các thành phần cấu thành

We discussed the component integration role earlier. Python's ability to be extended by and embedded in systems coded in C, C++, and Java makes it useful as a flexible glue language for scripting the behavior of other software components. For instance, integrating a C library into Python allows Python to test and launch the library's tools, and embedding Python in a product enables on-site customizations to be coded without having to recompile the entire product (or ship its source code at all).

Chúng ta đã thảo luận về vai trò của việc tích hợp các thành phần phần mềm vào hệ thống trước đây. Khả năng mở rộng và tích hợp của Python với các hệ thống được lập trình bằng ngôn ngữ C, C++ và Java khiến Python trở thành ngôn ngữ lý tưởng để lập trình các chức năng của các thành phần phần mềm khác. Ví dụ, việc tích hợp một thư viện viết bằng ngôn ngữ C vào Python giúp Python có thể kiểm thử và sử dụng các công cụ trong thư viện đó. Việc tích hợp Python vào một sản phẩm phần mềm cũng giúp việc tùy biến sản phẩm trở nên dễ dàng hơn, mà không cần phải biên dịch lại toàn bộ sản phẩm (hoặc gửi mã nguồn của sản phẩm đi cùng).

Tools such as *SWIG*, *Boost.Python*, *CFFI*, and *HPy* automate much of the work needed to link compiled components with Python scripts, and the *Cython* system allows coders to mix Python and C-like code to create compiled extensions. Other tools provide more ways to script components—including the *pyjnius* and *Chaquopy* Python/Java bridges; *pywin32*'s support for Windows Component Object Model (COM); the

REST, *SOAP*, and *XML-RPC* cross-network conduits; and the *Jython* Java and *IronPython* .NET implementations of Python. In short, most software components are in scope to Python code.

Các công cụ như SWIG, Boost.Python, CFFI và HPy giúp tự động hóa phần lớn các công việc cần thiết để kết hợp các thành phần đã được biên dịch với các chương trình viết bằng ngôn ngữ Python. Hệ thống Cython cho phép các lập trình viên kết hợp mã nguồn viết bằng Python với mã nguồn viết bằng ngôn ngữ C để tạo ra các thành phần có thể được sử dụng trong chương trình Python. Còn có nhiều công cụ khác giúp việc lập trình các thành phần này trở nên dễ dàng hơn – chẳng hạn như pyjnius và Chaquopy, những công cụ giúp kết nối Python với Java; pywin32, công cụ hỗ trợ việc sử dụng các thành phần của hệ điều hành Windows; các giao thức truyền dữ liệu qua mạng như REST, SOAP và XML-RPC; cùng các phiên bản Python được triển khai trên nền tảng Java của Jython và trên nền tảng .NET của IronPython. Nói chung, hầu hết các thành phần phần mềm đều có thể được sử dụng trong các chương trình viết bằng ngôn ngữ Python.

Database Access Truy cập cơ sở dữ liệu

For traditional database demands, there are Python interfaces to all commonly used relational database systems—Oracle, MySQL, PostgreSQL, Informix, ODBC, SQLite, and more. The Python world has also defined a *portable database API* for accessing SQL database systems from Python scripts, which looks the same on a variety of underlying database implementations. For basic program-storage needs, Python comes with built-in support for its own *pickle* objects, language-agnostic *JSON* documents, and the in-process *SQLite* embedded SQL database engine.

Đối với các nhu cầu liên quan đến cơ sở dữ liệu truyền thống, Python cung cấp các giao diện để kết nối với tất cả các hệ thống cơ sở dữ liệu quan hệ phổ biến hiện nay: Oracle, MySQL, PostgreSQL, Informix, ODBC, SQLite, v.v. Thế giới Python còn định nghĩa một giao diện API tiêu chuẩn để truy cập các hệ thống cơ sở dữ liệu SQL từ các chương trình viết bằng Python. Giao diện này hoạt động tương tự trên nhiều loại hệ thống cơ sở dữ liệu khác nhau. Đối với các nhu cầu lưu trữ dữ liệu cơ bản, Python có sẵn các công cụ hỗ trợ như đối tượng pickle, các tệp dữ liệu dạng JSON, và động cơ cơ sở dữ liệu SQLite tích hợp sẵn trong chương trình.

Also for Python scripts, *PyYAML* parses and emits YAML data; *ZODB* and *Durus* provide object-oriented databases; *SQLObject* and *SQLAlchemy* implement object relational mappers (ORMs), which graft Python classes onto relational tables; and *PyMongo* interfaces to MongoDB, a high-performance JSON-style document database. Python can also access cloud storage options in Google's *App Engine*, Microsoft's *Azure*, and Amazon's *AWS*.

Đối với các chương trình viết bằng Python, PyYAML có chức năng phân tích và xử lý dữ liệu dạng YAML. ZODB và Durus là những công cụ hỗ trợ việc quản lý dữ liệu theo hướng đối tượng. SQLObject và SQLAlchemy là các công cụ hỗ trợ việc chuyển đổi dữ liệu từ dạng đối tượng sang dạng bảng quan hệ; chúng giúp kết nối các lớp trong Python với các bảng quan hệ trong cơ sở dữ liệu. PyMongo lại là công cụ giúp kết nối Python với MongoDB – một cơ sở dữ liệu dạng tài liệu, hoạt động hiệu suất cao. Python cũng có thể truy cập các dịch vụ lưu trữ đám mây như App Engine của Google, Azure của Microsoft và AWS của Amazon.

Rapid Prototyping Tạo mẫu nhanh

To Python programs, components written in Python and C look the same. Because of this, it's possible to prototype systems in Python initially, and then move selected components to a fully compiled language such as C or C++ for delivery. Because Python doesn't require a complete rewrite once the prototype has solidified, the parts of the system that don't need the efficiency of a compiled language can remain coded in Python for ease of maintenance and use. But beware: given the many optimization routes you met earlier, your prototype may very well be your deliverable.

Đối với các chương trình viết bằng Python, các thành phần được viết bằng Python và C trông giống hệt nhau. Nhờ đó, người ta có thể tạo ra bản nguyên mẫu của hệ thống bằng Python trước, sau đó chuyển các thành phần cần thiết sang ngôn ngữ biên dịch như C hoặc C++ để hoàn thiện sản phẩm. Vì Python không yêu cầu phải viết lại toàn bộ mã nguồn sau khi bản nguyên mẫu được hoàn thiện, nên những phần của hệ thống không cần đến hiệu suất cao của ngôn ngữ biên dịch vẫn có thể được viết bằng Python, nhằm thuận tiện cho việc bảo trì và sử dụng. Nhưng hãy cẩn thận: với rất nhiều cách tối ưu hóa khác nhau, bản nguyên mẫu của bạn hoàn toàn có thể trở thành sản phẩm cuối cùng.

Numeric and Scientific Programming

Lập trình số học và khoa học

Python is also widely used in numeric programming—a domain that would not traditionally have been considered to be in the scope of scripting languages but has grown to become one of Python's most compelling use cases. Prominent here, the *NumPy* high-performance numeric-programming extension for Python mentioned earlier includes such advanced tools as an array object, interfaces to standard mathematical libraries, and much more. By integrating Python with numeric routines coded in a compiled language for speed, NumPy turns Python into a sophisticated yet easy-to-use numeric-programming tool that can often replace code written in traditional languages like Fortran or C++.

Python cũng được sử dụng rộng rãi trong lĩnh vực lập trình số học. Đây là lĩnh vực vốn không được coi là thuộc phạm vi của các ngôn ngữ lập trình kịch bản truyền thống. Tuy nhiên, ngày nay, lĩnh vực này đã trở thành một trong những ứng dụng nổi bật của Python. Trong số các công cụ hỗ trợ lập trình số học dành cho Python, có thể kể đến NumPy – một công cụ hiệu năng cao. NumPy cung cấp nhiều tính năng tiên tiến như các cấu trúc dữ liệu dạng mảng, các giao diện để truy cập vào các thư viện toán học tiêu chuẩn, v.v. Bằng cách kết hợp Python với các chương trình số học được viết bằng ngôn ngữ biên dịch nhằm tăng tốc độ thực thi, NumPy biến Python thành công cụ lập trình số học vừa mạnh mẽ vừa dễ sử dụng. Công cụ này thường có thể thay thế được các đoạn mã được viết bằng các ngôn ngữ truyền thống như Fortran hay C++.

Additional numeric tools often use NumPy as a core component, and add support for visualization, parallel processing, statistical analysis, and more. For example, *SciPy* provides libraries of scientific programming tools; *pandas* supports data analysis; *matplotlib* adds visualization tools; *Jupyter* notebooks are geared toward math workers' needs; *Numba* and *PyThran* offer just-in-time (JIT) and ahead-of-time (AOT) compilers for Python numeric code, respectively; and the *Cython* and *PyPy* systems noted earlier can optimize algorithmic code. For more basic needs, Python itself comes with a statistics module; complex, fixed-point, and rational math; and other tools you'll learn about in this book.

Các công cụ số học bổ sung thường sử dụng NumPy làm thành phần cốt lõi. Ngoài ra, chúng còn hỗ trợ các tính năng như trực quan hóa dữ liệu, xử lý song song, phân tích thống kê, v.v. Ví dụ, SciPy cung cấp các thư viện dùng để lập trình khoa học; pandas hỗ trợ việc phân tích dữ liệu; matplotlib giúp trực quan hóa dữ liệu; Jupyter Notebook phù hợp với nhu cầu của các nhà toán học; Numba và PyThran lần lượt cung cấp các công cụ biên dịch mã số học của Python theo kiểu JIT và AOT. Các công cụ như Cython và PyPy cũng giúp tối ưu hóa mã nguồn. Đối với những nhu cầu cơ bản hơn, chính Python đã có sẵn các công cụ cần thiết như mô-đun phân tích thống kê, các công cụ xử lý số học phức tạp, v.v. Bạn sẽ được tìm hiểu thêm về những công cụ này trong cuốn sách này.

And More: AI, Games, Images, QA, Excel, Apps…

Và còn nhiều thứ nữa: Trí tuệ nhân tạo, Trò chơi, Hình ảnh, Kiểm thử chất lượng, Excel, Ứng dụng di động…

Python is commonly applied in far more domains and with far more tools than can possibly be covered here. For example, it's also used in:

Python thường được ứng dụng trong nhiều lĩnh vực và với nhiều công cụ khác nhau, nhiều đến mức không thể liệt kê hết được ở đây. Ví dụ, Python còn được sử dụng trong:

- Artificial intelligence (see *PyTorch*, *TensorFlow*, and *Keras*)

  Trí tuệ nhân tạo (xem PyTorch, TensorFlow và Keras)

- Game programming (see *pygame*, *Panda3D*, and *Kivy*)

  Lập trình trò chơi (xem pygame, Panda3D và Kivy)

- Image and graphics processing (see *Pillow*, *PyOpenGL*, and *OpenCV*)

Xử lý hình ảnh và đồ họa (xem các thư viện Pillow, PyOpenGL và OpenCV)

- Quality assurance and testing (see *PyTest*, *unittest*, and *Selenium*)

  Đảm bảo chất lượng và kiểm thử (xem PyTest, unittest và Selenium)

- Excel spreadsheets (see *xlwings*, *PyXLL*, and *Excel*)

  Các bảng tính Excel (xem xlwings, PyXLL và Excel)

- Smartphone apps (see *Kivy*, *BeeWare*, and Appendix A)

  Ứng dụng trên điện thoại thông minh (xem Kivy, BeeWare và Phụ lục A)

- Microcontrollers and ports (see *MicroPython* and *PySerial*)

  Bộ điều khiển vi mạch và các cổng giao tiếp (xem MicroPython và PySerial)

- And of course, much more

  Và tất nhiên, còn nhiều thứ khác nữa.

For links to resources in these and many other fields, try Wikipedia's Python software page; the PyPI website, which hosts extension packages installed by Python's *pip*; and the normal web searches.

Để tìm các nguồn tài nguyên liên quan đến các lĩnh vực này và nhiều lĩnh vực khác nữa, bạn có thể tham khảo trang thông tin về phần mềm Python trên Wikipedia; trang web PyPI, nơi lưu trữ các gói mở rộng được cài đặt bằng công cụ pip của Python; cũng như các kết quả tìm kiếm trên mạng.

Though application domains underscore Python's practical utility, keep in mind that many are largely just instances of Python's component *integration* role in action again. Adding Python as a frontend to libraries of components written in a compiled language like C makes Python useful for scripting in a wide variety of roles. As a general-purpose language that supports integration, Python is broadly, if not universally, applicable.

Mặc dù các lĩnh vực ứng dụng cho thấy tính hữu ích thực tế của Python, nhưng cần lưu ý rằng nhiều trường hợp đó thực chất chỉ là ví dụ minh họa cho khả năng tích hợp các thành phần khác nhau của Python mà thôi. Việc sử dụng Python như công cụ lập trình phía frontend cho các thư viện được viết bằng ngôn ngữ biên dịch như C giúp Python trở thành công cụ hữu ích cho nhiều mục đích lập trình khác nhau. Là một ngôn ngữ đa năng, hỗ trợ việc tích hợp các thành phần khác nhau, Python có thể được áp dụng trong rất nhiều lĩnh vực, nếu không muốn nói là trong mọi lĩnh vực.

What Are Python's Technical Strengths?

Những điểm mạnh về mặt kỹ thuật của Python là gì?

Naturally, this is a developer's question. If you don't already have a programming background, the terminology in the next few sections may be a bit baffling—don't worry, we'll explore all of these topics in more detail as we proceed through this book. For both current and future developers, though, here is a quick rundown of Python's top technical features. Some have been touched on earlier, but this section fills in more of the story.

Tất nhiên, đây là những câu hỏi dành cho các lập trình viên. Nếu bạn chưa có kiến thức lập trình trước đó, có lẽ các thuật ngữ được sử dụng trong các phần tiếp theo sẽ hơi khó hiểu. Đừng lo lắng, chúng tôi sẽ giải thích chi tiết về tất cả các chủ đề này trong quá trình bạn đọc cuốn sách này. Dù bạn là lập trình viên hiện tại hay tương lai, dưới đây là cái nhìn tổng quan về các tính năng kỹ thuật chính của Python. Một số tính năng đã được đề cập ở phần trước, nhưng phần này sẽ giúp bạn hiểu rõ hơn về chúng.

It's Object-Oriented and Functional

Đây là phương pháp lập trình hướng đối tượng và hướng chức năng.

Python is an object-oriented language, from the ground up. As you'll find in this book, its *class model* supports advanced notions such as polymorphism, operator overloading, and multiple inheritance; yet, in the context of Python's simple syntax and typing, OOP is remarkably easy to apply. In fact, if you don't understand these terms, you'll find they are much easier to learn with Python than with just about any other OOP language available.

Python là một ngôn ngữ lập trình hướng đối tượng, ngay từ khởi đầu. Như bạn sẽ thấy trong cuốn sách này, mô hình lớp trong Python hỗ trợ các khái niệm nâng cao như đa hình, gán chức năng cho toán tử, và kế thừa đa cấp. Tuy nhiên, nhờ cú pháp và cách nhập dữ liệu đơn giản của Python, việc áp dụng các nguyên lý lập trình hướng đối tượng trở nên rất dễ dàng. Thực tế, nếu bạn chưa hiểu rõ các thuật ngữ này, bạn sẽ thấy rằng việc học chúng trong Python dễ dàng hơn nhiều so với việc học chúng trong bất kỳ ngôn ngữ lập trình hướng đối tượng nào khác.

Of equal significance, OOP is an *option* in Python; you can go far without having to become an object guru all at once. Much like C++, Python supports both procedural and object-oriented programming modes. Its object-oriented tools can be applied if and when constraints allow. This is especially useful in tactical development modes, which often preclude the design phases that best utilize OOP's benefits.

Cũng quan trọng không kém, OOP là một tính năng có sẵn trong Python. Bạn hoàn toàn có thể sử dụng OOP mà không cần phải trở thành chuyên gia về lập trình hướng đối tượng ngay lập tức. Giống như C++, Python hỗ trợ cả hai phương thức lập trình: lập trình thủ tục và lập trình hướng đối tượng. Các công cụ hỗ trợ lập trình hướng đối tượng trong Python có thể được sử dụng khi điều kiện cho phép. Điều này đặc biệt hữu ích trong các giai đoạn phát triển cụ thể, nơi mà việc áp dụng các nguyên lý của lập trình hướng đối tượng đôi khi bị hạn chế.

In addition to its original *procedural* (statement-based) and *object-oriented* (class-based) paradigms, Python today has built-in support for *functional* programming—a set that by most measures includes generators, comprehensions, closures, maps, decorators, anonymous-function lambdas, and first-class function objects. As you'll also learn in this book, these can serve as both complement and alternative to its OOP tools.

Ngoài các mô hình lập trình truyền thống như mô hình dựa trên các câu lệnh và mô hình hướng đối tượng, ngày nay Python còn hỗ trợ cả lập trình hàm. Các yếu tố cấu thành của lập trình hàm bao gồm các hàm sinh ra dữ liệu, các cấu trúc dữ liệu được tạo ra từ các câu lệnh, các hàm ẩn danh, các hàm được sử dụng để xử lý dữ liệu, v.v. Như bạn sẽ được tìm hiểu trong cuốn sách này, những yếu tố này có thể được sử dụng như những công cụ bổ trợ hoặc thay thế cho các công cụ lập trình hướng đối tượng trong Python.

It's Free and Open

Đây là dịch vụ miễn phí và có thể sử dụng một cách tự do.

Python is completely free to use and distribute. As with other *open source software*, such as Linux and Apache, you can fetch the entire Python system's source code for free on the internet. There are no restrictions on copying it, embedding it in your systems, or shipping it with your products.

Python hoàn toàn miễn phí để sử dụng và phân phối. Cũng giống như các phần mềm mã nguồn mở khác như Linux và Apache, bạn có thể tải về toàn bộ mã nguồn của Python từ Internet một cách miễn phí. Không có bất kỳ hạn chế nào đối với việc sao chép, tích hợp Python vào các hệ thống của bạn, hoặc đưa Python vào các sản phẩm của mình.

But don't get the wrong idea: "free" doesn't mean "unsupported." On the contrary, Python users have access to numerous online resources that respond to queries and issues quicker than most commercial software. Moreover, because Python comes with complete source code, it empowers developers in ways that closed commercial software cannot. Although studying or changing a programming language's implementation code isn't everyone's idea of fun, it's comforting to know that you can. You're not dependent on the whims of a commercial vendor, because the ultimate documentation—*source code*—is at your disposal as a last resort.

Nhưng đừng hiểu lầm: "miễn phí" không có nghĩa là "không được hỗ trợ". Ngược lại, người dùng Python có thể tiếp cận nhiều nguồn tài nguyên trực tuyến, những nguồn này có thể trả lời các thắc mắc và giải quyết các vấn đề một cách nhanh chóng hơn so với hầu hết các phần mềm thương mại. Hơn nữa, vì Python sở hữu mã nguồn hoàn chỉnh, nên nó mang lại nhiều lợi thế cho các lập trình viên, điều mà các phần mềm thương mại không thể làm được. Mặc dù việc nghiên cứu hoặc thay đổi mã nguồn của một ngôn ngữ lập trình không phải là điều mà ai cũng thích, nhưng việc biết rằng mình có thể làm điều đó thật sự rất an tâm. Bạn không cần phải phụ thuộc vào ý muốn của các nhà cung cấp phần mềm thương mại, bởi vì mã nguồn luôn sẵn sàng để bạn sử dụng trong trường hợp cần thiết.

It’s Portable Nó có thể được sử dụng một cách linh hoạt.

We touched on portability earlier. The standard implementation of Python is written in portable ANSI C, and compiles and runs on virtually every major platform currently in use. For example, Python programs run today on everything from smartphones to supercomputers. As a partial list, Python is available on Windows and macOS PCs, Linux and Unix workstations and servers, Android and iOS smartphones and tablets, real-time systems like VxWorks, Cray supercomputers, IBM mainframes, and more. Moreover, this list expands regularly; Android and iOS, for example, are gaining official *python.org* support at this writing.

Chúng ta đã đề cập đến khả năng di động của Python trước đây. Phiên bản tiêu chuẩn của Python được viết bằng ngôn ngữ lập trình ANSI C, vì vậy nó có thể được biên dịch và chạy trên hầu hết các nền tảng phổ biến hiện nay. Ví dụ, các chương trình viết bằng Python có thể chạy trên mọi thiết bị, từ điện thoại thông minh cho đến máy tính siêu mạnh. Python có thể được sử dụng trên Windows, macOS, Linux, Unix, cũng như trên các thiết bị di động như Android và iOS. Ngoài ra, Python còn có thể chạy trên các hệ thống thời gian thực như VxWorks, các máy tính siêu mạnh của Cray, các máy chủ IBM, và nhiều nền tảng khác nữa. Danh sách các nền tảng mà Python có thể chạy trên vẫn đang tiếp tục được mở rộng. Hiện tại, cả Android và iOS đều đã được hỗ trợ chính thức bởi trang web python.org.

Like the language itself, the *standard-library* modules that ship with Python are designed to be portable across platform boundaries; their file tools, for instance, remove many or most of the proprietary aspects of storage on some hosts. Furthermore, Python programs are automatically compiled to portable *byte-code*, which runs the same on any platform with a compatible version of Python installed (more on this in the next chapter), and there are multiple ways to code portable *user interfaces* in Python with traditional GUIs and web-based options described earlier.

Cũng giống như ngôn ngữ lập trình Python 本身, các mô-đun trong thư viện tiêu chuẩn đi kèm với Python cũng được thiết kế để có thể hoạt động trên nhiều nền tảng khác nhau. Chẳng hạn, các công cụ xử lý tệp tin trong Python loại bỏ phần lớn những yếu tố đặc thù liên quan đến cách lưu trữ dữ liệu trên một số nền tảng nhất định. Hơn nữa, các chương trình viết bằng Python sẽ được biên dịch thành dạng bytecode có thể chạy trên bất kỳ nền tảng nào có cài đặt phiên bản Python tương thích (chi tiết hơn về điều này sẽ được trình bày trong chương tiếp theo). Ngoài ra, còn có nhiều cách để tạo ra các giao diện người dùng có thể sử dụng trên nhiều nền tảng, bao gồm cả các giao diện GUI truyền thống và các giao diện dựa trên web.

This means that programs that use the Python language, its standard libraries, and portable extensions run the same on most systems that host a Python interpreter. Python also supports platform-specific extensions (e.g., *pywin32* on Windows, *PyObjC* on macOS, and *pyjnius* on Android), but Python itself works the same everywhere.

Điều này có nghĩa là các chương trình sử dụng ngôn ngữ Python, các thư viện tiêu chuẩn của nó, cũng như các tiện ích mở rộng khác, đều có thể chạy trên hầu hết các hệ thống có trình thông dịch Python. Python cũng hỗ trợ các tiện ích mở rộng dành riêng cho từng nền tảng cụ thể (ví dụ: pywin32 trên Windows, PyObjC trên macOS, pyjnius trên Android). Tuy nhiên, bản thân Python vẫn hoạt động giống nhau trên mọi nền tảng.

It’s Powerful Nó thật mạnh mẽ.

From a features perspective, Python is something of a hybrid. Its toolset places it between traditional scripting languages (such as Tcl, Scheme, and Perl) and systems development languages (such as C, C++, and Java). Python provides all the simplicity and ease of use of a scripting language, along with more advanced software-engineering tools typically found in compiled languages. Unlike some scripting languages, this combination makes Python useful for large-scale development projects. As a preview, here are some of the main things you'll find in Python's toolbox:

Xét về các tính năng, Python có thể được coi là một ngôn ngữ lập trình lai. Bộ công cụ mà Python cung cấp nằm giữa các ngôn ngữ lập trình kịch bản truyền thống (như Tcl, Scheme, Perl) và các ngôn ngữ lập trình hệ thống (như C, C++, Java). Python mang lại sự đơn giản và dễ sử dụng như các ngôn ngữ lập trình kịch bản, đồng thời cũng có những công cụ lập trình chuyên nghiệp thường thấy trong các ngôn ngữ lập trình biên dịch. Khác với một số ngôn ngữ lập trình kịch bản khác, sự kết hợp này khiến Python trở thành công cụ lý tưởng cho các dự án phát triển quy mô lớn. Dưới đây là một số tính năng chính của Python:

Dynamic typing *Kiểu dữ liệu động*

Python keeps track of the kinds of objects your program uses when it runs, and doesn't require complicated type and size declarations in your code. In fact, as you'll see in Chapter 6, there is no such thing as a type or variable declaration anywhere in Python (apart from recent "hinting," which Python itself does not use). Because Python code does not constrain data types, it is also usually automatically applicable to a whole range of objects.

Python ghi nhận các loại đối tượng mà chương trình của bạn sử dụng trong quá trình chạy. Bạn không cần phải khai báo các kiểu dữ liệu hay kích thước của các biến một cách phức tạp trong mã nguồn. Thực tế, như bạn sẽ thấy trong Chương 6, trong Python không hề có khái niệm về việc khai báo kiểu dữ liệu hay biến nào cả (ngoại trừ một vài trường hợp đặc biệt mà chính Python cũng không sử dụng). Vì mã nguồn Python không giới hạn các kiểu dữ liệu, nên nó có thể được áp dụng một cách tự động cho nhiều loại đối tượng khác nhau.

Automatic memory management

Quản lý bộ nhớ tự động

Python automatically allocates objects and reclaims ("garbage collects") them when they are no longer used. As you'll learn, Python keeps track of objects in use and the memory they hold, so you don't have to.

Python tự động cấp phát bộ nhớ cho các đối tượng và thu hồi bộ nhớ đó khi chúng không còn được sử dụng nữa. Như bạn sẽ biết, Python luôn theo dõi các đối tượng đang được sử dụng và lượng bộ nhớ mà chúng chiếm giữ; vì vậy, bạn không cần phải làm điều đó.

Programming-in-the-large support

Hỗ trợ việc lập trình trên quy mô lớn

For building larger systems, Python includes tools such as modules, classes, and exceptions. These tools allow you to organize systems into components, use OOP to reuse and customize code, and handle events and errors gracefully. Python's functional programming tools, described earlier, meet some of the same goals.

Để xây dựng các hệ thống lớn hơn, Python cung cấp nhiều công cụ như các mô-đun, lớp và cơ chế xử lý ngoại lệ. Những công cụ này giúp người dùng chia nhỏ hệ thống thành các thành phần riêng biệt, tận dụng nguyên lý lập trình hướng đối tượng để tái sử dụng và tùy chỉnh mã nguồn, đồng thời xử lý các sự kiện và lỗi một cách hiệu quả. Các công cụ lập trình hàm trong Python cũng giúp đạt được những mục tiêu tương tự.

Built-in object types *Các kiểu đối tượng được xây dựng sẵn*

Python provides commonly used data structures such as lists, dictionaries, and strings as intrinsic parts of the language. As you'll see, its built-in objects are both flexible and easy to use. They can grow and shrink on demand, can be arbitrarily nested to represent complex information, and are immune to common memory errors.

Python cung cấp các cấu trúc dữ liệu thường được sử dụng như danh sách, từ điển và chuỗi ký tự, và những cấu trúc này được coi là những thành phần cơ bản của ngôn ngữ lập trình này. Các đối tượng được xây dựng sẵn trong Python rất linh hoạt và dễ sử dụng. Chúng có thể được mở rộng hoặc thu hẹp tùy theo nhu cầu, có thể được sử dụng để biểu diễn các thông tin phức tạp, và kháng được các lỗi liên quan đến bộ nhớ.

Built-in tools *Các công cụ được tích hợp sẵn*

To process all those object types, Python comes with powerful and standard operations, including concatenation (joining collections), slicing (extracting sections), sorting, mapping, and more.

Để xử lý tất cả các loại đối tượng đó, Python cung cấp nhiều thao tác mạnh mẽ và tiêu chuẩn, bao gồm việc kết hợp các đối tượng lại với nhau, trích xuất các phần cần thiết từ đối tượng, sắp xếp dữ liệu, áp dụng các phép biến đổi lên dữ liệu, v.v.

Library utilities *Các tiện ích liên quan đến thư viện*

For more specific tasks, Python also comes with a large collection of precoded library tools that support everything from regular expression pattern matching to network servers. Once you learn the language itself, Python's library tools are where much of the application-level action occurs.

Đối với các nhiệm vụ cụ thể hơn, Python còn cung cấp nhiều công cụ thư viện đã được lập trình sẵn. Những công cụ này hỗ trợ mọi thứ, từ việc xử lý các mẫu ký tự thông thường cho đến việc quản lý các máy chủ mạng. Một khi bạn đã nắm vững ngôn ngữ lập trình Python, các công cụ thư viện này chính là nền tảng để thực hiện các chức năng cần thiết trong ứng dụng.

Third-party utilities *Các tiện ích của bên thứ ba*

Because Python is open source, developers are encouraged to contribute precoded tools that support tasks beyond those supported by its built-ins. On the Web, you'll find free support for image processing, numeric programming, database access, website development, formal testing, and much more (see “What Can I Do with Python?”).

Vì Python là phần mềm mã nguồn mở, nên các nhà phát triển được khuyến khích đóng góp các công cụ đã được lập trình sẵn, nhằm hỗ trợ những công việc mà các tính năng có sẵn trong Python chưa thể thực hiện được. Trên mạng Internet, bạn có thể tìm thấy nhiều công cụ hữu ích miễn phí để xử lý hình ảnh, lập trình số, truy cập cơ sở dữ liệu, phát triển trang web, kiểm thử chương trình, v.v. (Xem mục “Tôi có thể làm gì với Python?”)

Despite the array of tools in Python, it retains a noticeably simple syntax and design. The result is a powerful programming tool with all the usability of a scripting language.

Mặc dù có nhiều công cụ hỗ trợ lập trình trong Python, ngôn ngữ này vẫn giữ được cú pháp và cấu trúc đơn giản. Kết quả là, Python trở thành một công cụ lập trình mạnh mẽ, đồng thời vẫn giữ được tính tiện dụng của một ngôn ngữ lập trình kịch bản.

It's Mixable Nó có thể được trộn lẫn với các chất khác.

As noted earlier, Python programs can easily be “glued” to components written in other languages in a variety of ways—both locally and across networks. That means you can add functionality to the Python system as needed and use Python programs within other environments or systems.

Như đã đề cập trước đó, các chương trình viết bằng Python có thể dễ dàng được kết hợp với các thành phần được viết bằng các ngôn ngữ khác, theo nhiều cách khác nhau – cả ở cấp độ cục bộ lẫn qua mạng. Điều này có nghĩa là bạn có thể bổ sung các chức năng cần thiết vào hệ thống Python, đồng thời sử dụng các chương trình Python trong các môi trường hoặc hệ thống khác.

Mixing Python into systems coded in more demanding languages, for instance, makes it an easy-to-use frontend language for testing and customization. As also mentioned earlier, this makes Python good at rapid prototyping too—systems may be coded in Python first for development speed and later moved to extensions one piece at a time for execution speed if and when needed.

Việc kết hợp Python với các hệ thống được lập trình bằng các ngôn ngữ khác giúp Python trở thành ngôn ngữ lý tưởng để sử dụng trong các công việc kiểm thử và tùy biến hệ thống. Như đã đề cập trước đó, điều này cũng giúp Python trở thành công cụ tuyệt vời để tạo ra các phiên bản thử nghiệm nhanh chóng. Các hệ thống có thể được lập trình bằng Python trước để tăng tốc độ phát triển; sau đó, chúng có thể được chuyển sang các phiên bản hoàn chỉnh để chạy trên hệ thống thực tế, nếu cần thiết.

It's Relatively Easy to Use

Nó khá dễ sử dụng.

Compared to alternatives like C++, Java, and C#, Python programming seems astonishingly simple to most observers. To run Python code in most contexts, you simply type it and run it. There are no intermediate compile and link steps, like those typical for languages such as C or C++. Python executes programs immediately, which makes for an interactive programming experience and *rapid turnaround* after program changes—in many cases, you can witness the effect of a program change nearly as fast as you can type it.

So với các ngôn ngữ lập trình khác như C++, Java và C#, Python có vẻ cực kỳ đơn giản đối với hầu hết mọi người. Để chạy mã Python, bạn chỉ cần gõ mã lên và thực thi nó. Không cần qua các bước biên dịch hay liên kết phức tạp như trong các ngôn ngữ như C hay C++. Python thực thi chương trình ngay lập tức, giúp mang lại trải nghiệm lập trình tương tác tốt hơn và giúp việc thay đổi mã chương trình diễn ra nhanh chóng. Trong nhiều trường hợp, bạn có thể thấy ngay hiệu quả của việc thay đổi mã chương trình, gần như ngay lập tức sau khi bạn gõ mã vào.

Of course, development cycle turnaround is only one aspect of Python's ease of use. It also provides a deliberately simple syntax and powerful built-in tools. In fact, some have gone so far as to call Python *executable pseudocode*. Because it eliminates much of the complexity of its contemporaries, Python programs are simpler, smaller, and more flexible than equivalent programs in other popular languages.

Tất nhiên, thời gian cần thiết để phát triển một chương trình bằng Python chỉ là một trong những yếu tố thể hiện sự dễ sử dụng của ngôn ngữ này. Python còn có cú pháp đơn giản và các công cụ hỗ trợ mạnh mẽ. Thực tế, có người còn coi Python như một dạng "mã giả". Nhờ loại bỏ phần lớn những yếu tố phức tạp so với các ngôn ngữ lập trình khác, các chương trình viết bằng Python trở nên đơn giản hơn, gọn nhẹ hơn và linh hoạt hơn so với các chương trình tương đương được viết bằng các ngôn ngữ khác.

Which is not to say that Python makes programming a *no-brainer*. Python also has convolutions and dark corners that we'll tackle head-on in this book, and some of its roles are more rapid than others. Python's flavor of OOP inheritance, for example, is much more complicated than it once was, and building standalone apps or precompiled programs of the sort you'll meet in the next chapter can still be slow. Measured by its peers, though, Python is dramatically more coder friendly.

Điều này không có nghĩa là Python khiến việc lập trình trở nên hiển nhiên. Python cũng có những phức tạp và góc tối mà chúng ta sẽ đối mặt trực tiếp trong cuốn sách này, và một số vai trò của nó diễn ra nhanh hơn những vai trò khác. Ví dụ, kiểu kế thừa OOP của Python giờ đây phức tạp hơn nhiều so với trước đây, và việc xây dựng ứng dụng độc lập hoặc chương trình biên dịch sẵn như chương tiếp theo vẫn có thể chậm chạp. Tuy nhiên, nếu so sánh với các đối thủ, Python thân thiện với lập trình viên hơn rất nhiều.

It's Relatively Easy to Learn

Việc học nó khá dễ dàng.

Finally, this brings us to the point of this book: especially when compared to other widely used programming languages, the core Python language is remarkably easy to learn. In fact, if you're already an experienced programmer, you can expect to be coding small-scale Python programs in a matter of days—though you shouldn't expect to become an expert quite that fast, despite what you may have heard from marketing departments!

Cuối cùng, điều này dẫn chúng ta đến mục đích của cuốn sách này: So với các ngôn ngữ lập trình khác được sử dụng rộng rãi, ngôn ngữ Python thực sự rất dễ học. Thực tế, nếu bạn đã là lập trình viên có kinh nghiệm, bạn có thể viết các chương trình nhỏ bằng Python trong vài ngày thôi. Tuy nhiên, bạn không nên kỳ vọng sẽ trở thành chuyên gia về Python nhanh đến vậy, dù có thể bạn đã nghe nhiều điều tích cực về ngôn ngữ này từ các bộ phận tiếp thị!

Naturally, mastering any topic as substantial as today's Python is not trivial, and we'll devote the rest of this book to this task. But the investment required to master Python is worthwhile: in the end, you'll gain programming skills that apply to nearly every computer application domain. Moreover, most find Python's learning curve to be much gentler than that of other programming tools, even if that curve is not quite as flat as some content publishers claim.

Tất nhiên, việc nắm vững một ngôn ngữ lập trình phức tạp như Python ngày nay không hề dễ dàng chút nào. Chúng tôi sẽ dành phần còn lại của cuốn sách này để trình bày chi tiết về việc học Python. Nhưng những nỗ lực bỏ ra để nắm vững Python thực sự xứng đáng: cuối cùng, bạn sẽ có được những kỹ năng lập trình có thể được áp dụng trong hầu hết các lĩnh vực ứng dụng máy tính. Hơn nữa, hầu hết mọi người đều thấy rằng độ khó khi học Python thấp hơn nhiều so với các ngôn ngữ lập trình khác, mặc dù độ khó đó không hề đơn giản như một số người tuyên bố.

That’s good news for both professional developers seeking to add the language to their toolbox, and end users of systems that expose a Python layer for scripting roles. Today, many systems rely on the fact that people can learn enough Python to use the system with little or no support. Moreover, Python has spawned a legion of users who program for need or fun instead of career and may never require full-scale software development skills. Although Python has advanced tools you’ll meet in this book, its core fundamentals are accessible to beginners and gurus alike.

Đó là tin tốt cho cả các nhà phát triển chuyên nghiệp muốn bổ sung ngôn ngữ lập trình này vào công cụ làm việc của mình, lẫn những người dùng cuối sử dụng các hệ thống có khả năng hỗ trợ việc lập trình bằng Python. Ngày nay, nhiều hệ thống hoạt động dựa trên giả định rằng người dùng có thể học đủ kiến thức về Python để sử dụng hệ thống đó mà không cần sự hỗ trợ nhiều. Hơn nữa, Python đã thu hút được rất nhiều người dùng – những người lập trình vì nhu cầu cá nhân hoặc vì niềm yêu thích, chứ không phải vì mục đích nghề nghiệp. Những người dùng này có lẽ sẽ không bao giờ cần đến những kỹ năng phát triển phần mềm chuyên nghiệp. Mặc dù Python có nhiều công cụ tiên tiến mà bạn sẽ được tìm hiểu trong cuốn sách này, nhưng những kiến thức cơ bản của Python vẫn dễ tiếp cận đối với cả người mới bắt đầu lẫn những chuyên gia lập trình.

To see all this for yourself, let’s wrap up this overview and get started coding.

Để tự mình tận mắt chứng kiến tất cả những điều này, hãy kết thúc phần tổng quan này và bắt tay vào việc lập trình thôi.

Chapter Summary Tóm tắt chương

And that concludes the “hype” portion of this book. In this chapter, you’ve explored some of the reasons that people pick Python for their programming tasks. You’ve also seen how it is applied and looked at a representative sample of notable users today. This book’s goal is to teach Python, though, not to sell it. The best way to judge a language is to see it in action, so the rest of this book focuses entirely on the language details glossed over here.

Và như vậy, phần giới thiệu về ngôn ngữ lập trình Python trong cuốn sách này đã kết thúc. Trong chương này, bạn đã tìm hiểu một số lý do tại sao mọi người chọn Python để thực hiện các công việc lập trình của mình. Bạn cũng đã biết cách sử dụng Python, đồng thời tìm hiểu về một số người dùng tiêu biểu của ngôn ngữ này hiện nay. Mục đích của cuốn sách này là giúp mọi người hiểu rõ hơn về Python, chứ không phải để quảng bá cho ngôn ngữ này. Cách tốt nhất để đánh giá một ngôn ngữ lập trình là xem cách nó được sử dụng trong thực tế. Vì vậy, phần còn lại của cuốn sách sẽ tập trung vào các chi tiết liên quan đến ngôn ngữ Python.

The next two chapters begin your technical introduction to the language. In them, you’ll study ways to run Python programs, peek at Python’s execution model, and learn the basics of module files for saving code. The aim will be to give you just enough information to run the examples and exercises in the rest of the book. You won’t really start programming per se until Chapter 4, but make sure you have a handle on the startup details before moving on.

Hai chương tiếp theo sẽ giúp bạn làm quen với các khía cạnh kỹ thuật của ngôn ngữ lập trình Python. Trong các chương này, bạn sẽ tìm hiểu cách chạy các chương trình viết bằng Python, tìm hiểu cách thức thực thi các chương trình Python, và học cách sử dụng các tệp mã nguồn để lưu trữ mã lập trình. Mục tiêu của các chương này là cung cấp cho bạn đủ thông tin cần thiết để có thể thực hiện các ví dụ và bài tập trong phần còn lại của cuốn sách. Bạn sẽ chưa thực sự bắt đầu lập trình cho đến Chương 4. Tuy nhiên, hãy chắc chắn rằng bạn đã nắm vững các thông tin cơ bản liên quan đến việc khởi động chương trình trước khi tiếp tục.

Test Your Knowledge: Quiz

Kiểm tra kiến thức của bạn: Bài trắc nghiệm

In this book, we will be closing each chapter with a quick open-book quiz about the chapter's coverage to help you review key concepts. The answers for these quizzes appear immediately after the questions, and you are encouraged to read the answers once you've taken a crack at the questions yourself, as they sometimes give useful summary context.

Trong cuốn sách này, chúng tôi sẽ kết thúc mỗi chương bằng một bài kiểm tra ngắn gọn nhằm giúp bạn ôn tập lại các kiến thức quan trọng trong chương đó. Các đáp án sẽ được hiển thị ngay sau các câu hỏi. Bạn nên đọc các đáp án sau khi tự trả lời các câu hỏi trước; đôi khi, các đáp án sẽ cung cấp những thông tin tổng kết hữu ích.

In addition to these end-of-chapter quizzes, you'll find lab *exercises* at the end of each *part* of the book, designed to help you start coding Python on your own, with suggested answers available in an appendix. For now, here's your first quiz. Good luck, and be sure to refer back to this chapter's material as needed.

Ngoài các bài kiểm tra cuối chương này, bạn còn sẽ tìm thấy các bài tập thực hành ở cuối mỗi phần của cuốn sách. Những bài tập này được thiết kế nhằm giúp bạn tự mình bắt đầu học lập trình Python. Các đáp án được đưa ra ở phần phụ lục. Đây là bài kiểm tra đầu tiên của bạn. Chúc bạn may mắn! Đừng quên tham khảo lại nội dung của chương này mỗi khi cần thiết.

1. What are the six main reasons that people choose to use Python?

   Đó là những lý do chính nào khiến mọi người chọn sử dụng Python?

2. Name four notable companies or organizations using Python today.

   Hãy nêu tên bốn công ty hoặc tổ chức nổi tiếng hiện đang sử dụng ngôn ngữ lập trình Python.

3. Why might you *not* want to use Python in an application?

   Tại sao bạn lại không muốn sử dụng Python trong một ứng dụng nào đó?

4. What can you do with Python?

   Bạn có thể làm gì với Python?

Test Your Knowledge: Answers

Kiểm tra kiến thức của bạn: Đáp án

How did you do? Here are the suggested answers, though there may be multiple solutions to some quiz questions. Again, even if you're sure of your answers, you're encouraged to look at the suggestions for additional context. See the chapter's coverage for more details if any of these responses don't make sense to you.

Bạn làm bài kiểm tra như thế nào? Dưới đây là các câu trả lời được đề xuất. Lưu ý rằng đối với một số câu hỏi, có thể có nhiều cách trả lời khác nhau. Ngay cả khi bạn chắc chắn về câu trả lời của mình, bạn vẫn nên xem các gợi ý được đưa ra để hiểu rõ hơn. Nếu có câu trả lời nào khiến bạn không hiểu, hãy xem phần giải thích chi tiết trong chương này.

1. Software quality, developer productivity, program portability, support libraries, component integration, and simple enjoyment. Of these, the quality and productivity themes seem to be the main reasons that people choose to use Python, but enjoyment counts, too, in a field that can be as challenging as software.

   Chất lượng phần mềm, năng suất của các lập trình viên, khả năng chuyển đổi chương trình sang các nền tảng khác nhau, các thư viện hỗ trợ, khả năng tích hợp các thành phần khác nhau của chương trình, và yếu tố "sự thích thú khi sử dụng phần mềm". Trong số những yếu tố này, chất lượng và năng suất dường như là những lý do chính khiến mọi người chọn sử dụng Python. Tuy nhiên, yếu tố "sự thích thú" cũng rất quan trọng trong lĩnh vực phần mềm, vốn đầy thách thức như vậy.

2. Google, Industrial Light & Magic, JPL, ESRI, Instagram, and many more. Almost every organization do-

ing software development uses Python in some fashion, whether for long-term strategic product development or for short-term tactical tasks such as testing and system administration.

Google, Industrial Light & Magic, JPL, ESRI, Instagram, và nhiều công ty khác nữa. Hầu như mọi tổ chức chuyên phát triển phần mềm đều sử dụng Python vào một mục đích nào đó, dù là để phát triển sản phẩm trong dài hạn hay để thực hiện các nhiệm vụ ngắn hạn như kiểm thử và quản trị hệ thống.

3. Python’s main downside is performance: in its currently most-common version, at least, it won’t run as quickly as fully compiled languages like C and C++. On the other hand, it’s quick enough for most applications, and typical Python code runs at close to C speed anyhow because it invokes linked-in C code in the interpreter. If speed is critical, compiled extensions and other tools like Cython are available to optimize the number-crunching parts of a Python application.

   Nhược điểm chính của Python là tốc độ thực thi: ít nhất là trong phiên bản phổ biến nhất hiện nay, Python không thể chạy nhanh bằng các ngôn ngữ được biên dịch hoàn toàn như C và C++. Tuy nhiên, đối với hầu hết các ứng dụng, tốc độ của Python vẫn đủ để sử dụng. Thông thường, mã Python chạy với tốc độ gần tương đương với mã C, vì Python gọi các đoạn mã C được tích hợp sẵn trong trình thông dịch của mình. Nếu tốc độ là yếu tố quan trọng, người dùng có thể sử dụng các công cụ như Cython để tối ưu hóa các phần của ứng dụng Python liên quan đến việc xử lý dữ liệu số.

4. You can use Python for nearly anything you can do with a computer, from website development and gaming to AI and spacecraft control. Numeric programming and web development may lead the pack today, though probably because those are some of the main things for which computers are used.

   Bạn có thể sử dụng Python để thực hiện hầu như mọi công việc có thể được thực hiện bằng máy tính: từ việc phát triển trang web, chơi trò chơi, cho đến ứng dụng trí tuệ nhân tạo và điều khiển tàu vũ trụ. Lập trình số và phát triển web hiện nay là hai lĩnh vực phổ biến nhất khi sử dụng Python. Có lẽ đó cũng chính là những lĩnh vực mà máy tính được sử dụng phổ biến nhất.